

## 4.4 — Implement Validation, Retry, and Feedback Loops

### Concept

Agents that call Claude once and trust the result unconditionally are fragile. Production-grade agentic systems treat model output as an unverified claim: they validate it against a schema or set of rules, and if it fails, they retry with a prompt that includes a precise description of what went wrong. This is the validation-retry pattern.

**Domain mental model:** Think of the agent as a compiler and Claude as a source file. The compiler does not ask the programmer to rewrite the entire file when one function has a type error — it reports the exact error and location. The retry prompt is your error report.

### The Validation-Retry Pattern

The core loop is:

1. Call Claude with the task prompt.
2. Parse and validate the output against a schema or set of assertions.
3. If valid, accept and proceed.
4. If invalid, construct a retry prompt that includes: the original task, the invalid output, and a precise description of the validation failure.
5. Repeat from step 1.

The critical distinction between naive retry and retry-with-error-feedback is step 4. Sending the original prompt again without error context is a coin flip — Claude may produce the same invalid output again. Including the exact failure gives the model a concrete problem to reason about and correct.

### Knowing When to Stop

Unlimited retries are a denial-of-service vector against your own system. Every retry loop must have a hard upper bound ( `maxAttempts` ). Beyond the budget, fail explicitly rather than silently degrade.

Diminishing returns set in fast. Empirically, if Claude cannot produce a valid output within three attempts given the same schema and progressively more specific error feedback, the underlying issue is usually one of:

- The schema is too strict or contradicts the task description.
- The task prompt is ambiguous in a way that consistently causes the same class of error.
- The required output format is outside the model's current capability for the given input.

In all three cases, retrying further wastes tokens without improving outcomes. The right response is to surface the failure to the caller, not to retry indefinitely.

### Tracking Detected Patterns to Avoid Loops

A subtler failure mode is a retry loop where each attempt produces a different invalid output, exhausting the attempt budget without ever surfacing a clear root cause. To detect this, track the validation error returned by each attempt alongside a fingerprint of the error type. If the same error type recurs across multiple attempts despite increasingly specific feedback, the pattern itself is the signal: stop retrying and escalate.

The `detected_pattern` field in the retry state accumulates error categories across attempts. When a category appears more than once, the agent knows the retry strategy is not converging and can adjust: broaden the schema, simplify the task prompt, or route to a fallback.

## Structured Validation with Schemas

Accept output only after it passes schema validation. Schemas serve two purposes: they define the contract precisely enough that validation errors are machine-readable, and they force you to specify the output format explicitly before implementation rather than discovering its shape ad hoc.

Use Zod (TypeScript) or equivalent schema libraries. Validate at the boundary — before the result touches any downstream code.

---

## Code Example

```
import { z } from "zod";
import Anthropic from "@anthropic-ai/sdk";

const client = new Anthropic();

// Define the expected output shape as a schema.
const ExtractedEventSchema = z.object({
  title: z.string().min(1),
  date: z.string().regex(/^\d{4}-\d{2}-\d{2}$/),
  location: z.string().optional(),
});

type ExtractedEvent = z.infer<typeof ExtractedEventSchema>;

interface RetryState {
  attempts: number;
  detected_pattern: Record<string, number>; // error type → occurrence count
}

async function extractEventWithRetry(
  rawText: string,
  maxAttempts = 3
): Promise<ExtractedEvent> {
  const state: RetryState = { attempts: 0, detected_pattern: {} };
  let lastError: string | null = null;
  let lastOutput: string | null = null;
```

```

while (state.attempts < maxAttempts) {
  state.attempts++;

  // Build the prompt. On retry, include the previous failure.
  const feedbackSection =
    lastError !== null
      ? `
\n\nYour previous attempt produced this output:\n${lastOutput}\n\nValidation failed
      : "";

  const prompt =
    `Extract the event details from the text below and respond with a single JSON object ` +
    `matching this schema: { title: string, date: "YYYY-MM-DD", location?: string }.\n\n` +
    `Text:\n${rawText}` +
    feedbackSection;

  const response = await client.messages.create({
    model: "claude-sonnet-4-5",
    max_tokens: 256,
    messages: [{ role: "user", content: prompt }],
  });

  const rawOutput =
    response.content[0].type === "text" ? response.content[0].text : "";
  lastOutput = rawOutput;

  // Parse JSON defensively before schema validation.
  let parsed: unknown;
  try {
    parsed = JSON.parse(rawOutput.trim());
  } catch {
    lastError = "Response was not valid JSON.";
    const errorKey = "invalid_json";
    state.detected_pattern[errorKey] =
      (state.detected_pattern[errorKey] ?? 0) + 1;

    if ((state.detected_pattern[errorKey] ?? 0) >= 2) {
      throw new Error(
        `Retry loop detected: model consistently produces invalid JSON after ${state.attempts}
      );
    }
    continue;
  }

  // Schema validation – produces structured, machine-readable errors.
  const result = ExtractedEventSchema.safeParse(parsed);
  if (result.success) {
    return result.data;
  }
}

```

```

    }

    // Convert Zod issues into a readable error string for the retry prompt.
    const issues = result.error.issues
      .map((i) => `${i.path.join(".")}: ${i.message}`)
      .join("; ");
    lastError = issues;

    // Track each error field as a pattern key to detect loops.
    for (const issue of result.error.issues) {
      const errorKey = `${issue.path.join(".")}_${issue.code}`;
      state.detected_pattern[errorKey] =
        (state.detected_pattern[errorKey] ?? 0) + 1;

      if ((state.detected_pattern[errorKey] ?? 0) >= 2) {
        throw new Error(
          `Retry loop detected on field "${issue.path.join(".")}" : ` +
            `same validation error after ${state.attempts} attempts. ` +
            `Review schema or task prompt.`
        );
      }
    }
  }

  throw new Error(
    `Output validation failed after ${maxAttempts} attempts. Last error: ${lastError}`
  );
}

```

The `detected_pattern` map accumulates error-type counts across attempts. When the same error appears twice, the agent stops retrying and throws a descriptive error rather than exhausting the full budget on a loop that is not converging.

---

## Common Mistakes

- 1. Retrying without error feedback** Sending the original prompt again without explaining what failed is the most common mistake. The model has no new information and will often reproduce the same output. Always include the invalid response and the specific validation error in the retry prompt.
- 2. No maximum attempt limit** An unbounded retry loop can run until it exhausts the token budget or triggers rate limits. Always set `maxAttempts` explicitly and fail with a clear error when the limit is reached.
- 3. Schema defined after implementation** Writing the prompt first and then discovering the output shape during validation means your schema is reverse-engineered from observed outputs. Define the schema before writing the prompt. The schema is the contract; the prompt should reference it explicitly.

**4. Treating all validation failures as equal** A missing optional field and a malformed date are different failure classes requiring different corrections. Log error types distinctly. Retry prompts that conflate distinct failures produce unfocused corrections.

**5. Ignoring diminishing returns** Three failed attempts with specific error feedback and no improvement is a strong signal that the retry strategy itself is wrong. Stop retrying. Audit the schema for over-constraints or the prompt for ambiguity before increasing `maxAttempts` .

**6. Not tracking patterns across retries** Without a `detected_pattern` record, a loop that cycles through different invalid outputs on each attempt can silently exhaust the attempt budget with no useful signal about root cause. Track error types across the full retry sequence, not just the most recent attempt.

---

## References

---

- Anthropic documentation: Tool use and structured output patterns
- Zod schema validation library: <https://zod.dev>
- Exam domain 4: Reliability and Production Patterns
- Related handouts: 4.1 (Tool Error Handling), 4.2 (Structured Output Contracts), 4.3 (Graceful Degradation)
- See also: 3.5 (Iterative Refinement Techniques) — retry-with-feedback is the automated equivalent of the structured feedback loop described there